

Stop Writing Specs for AI. Start Prototyping.

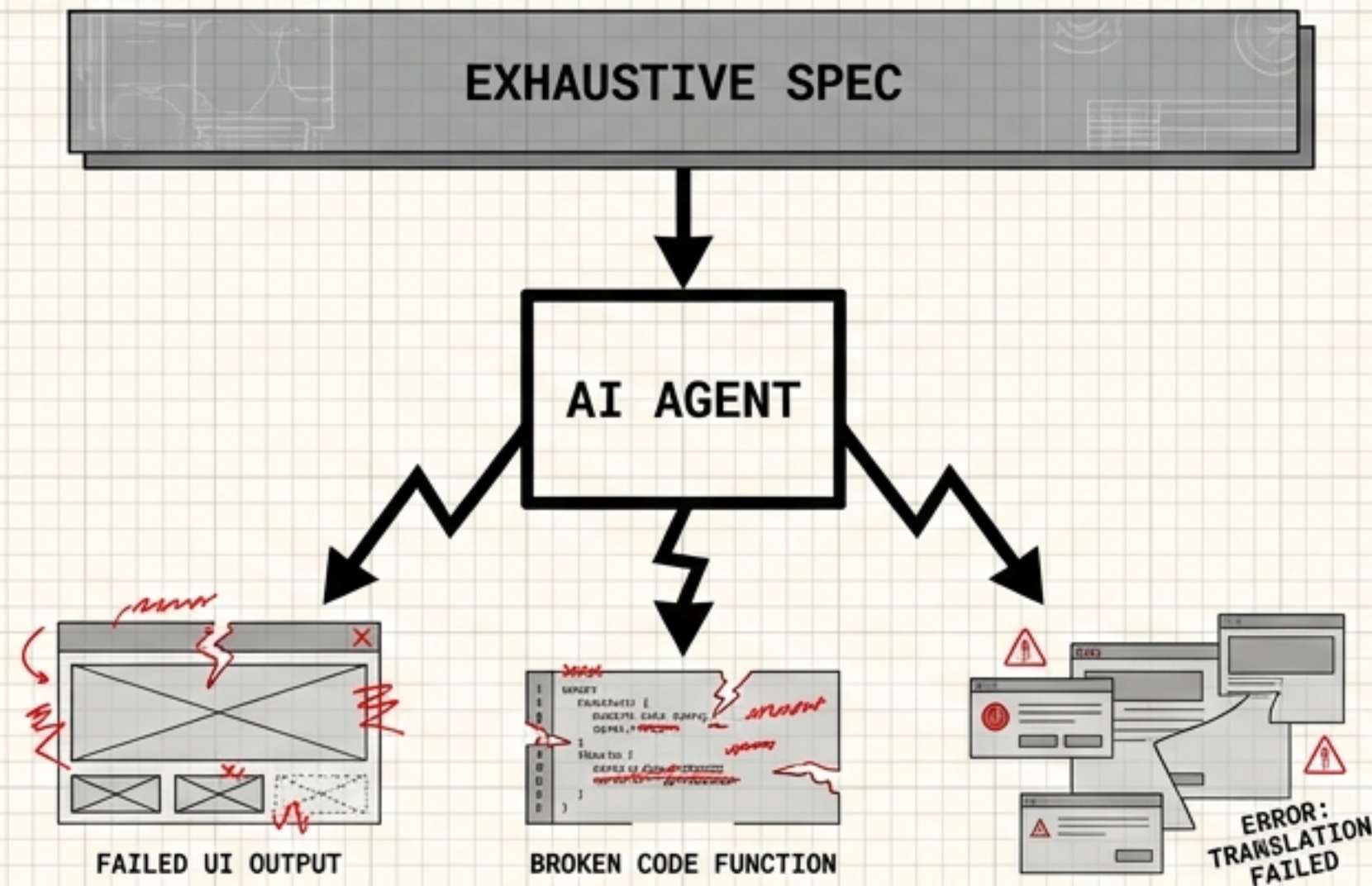
The economics of software have shifted. Why your planning medium must change from static text to executable code.



The Spec-Driven Trap

Developers treat **AI agents like compilers for human text**. We pour hours into writing massive, hyper-detailed specifications, expecting the AI to perfectly nail the output on the first try.

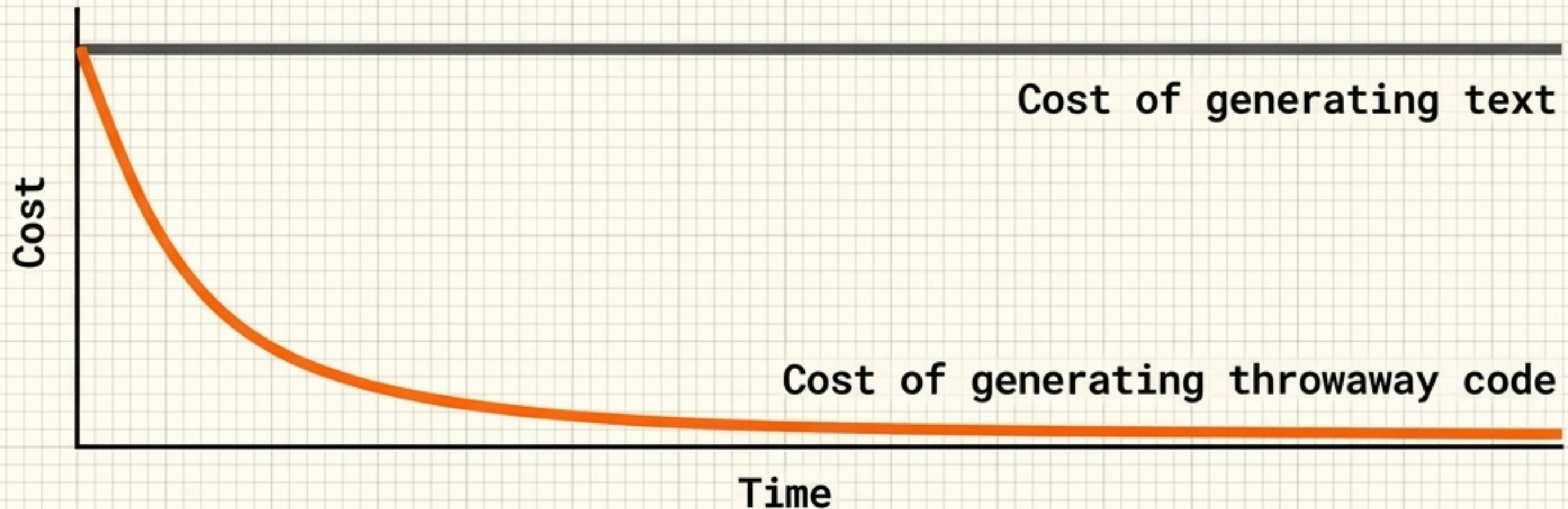
We carry over legacy planning habits, forgetting one fundamental truth: we are now equipped to write code while figuring out the spec.



The New Economics of Code

Producing prototypes and quick spikes has never been cheaper.

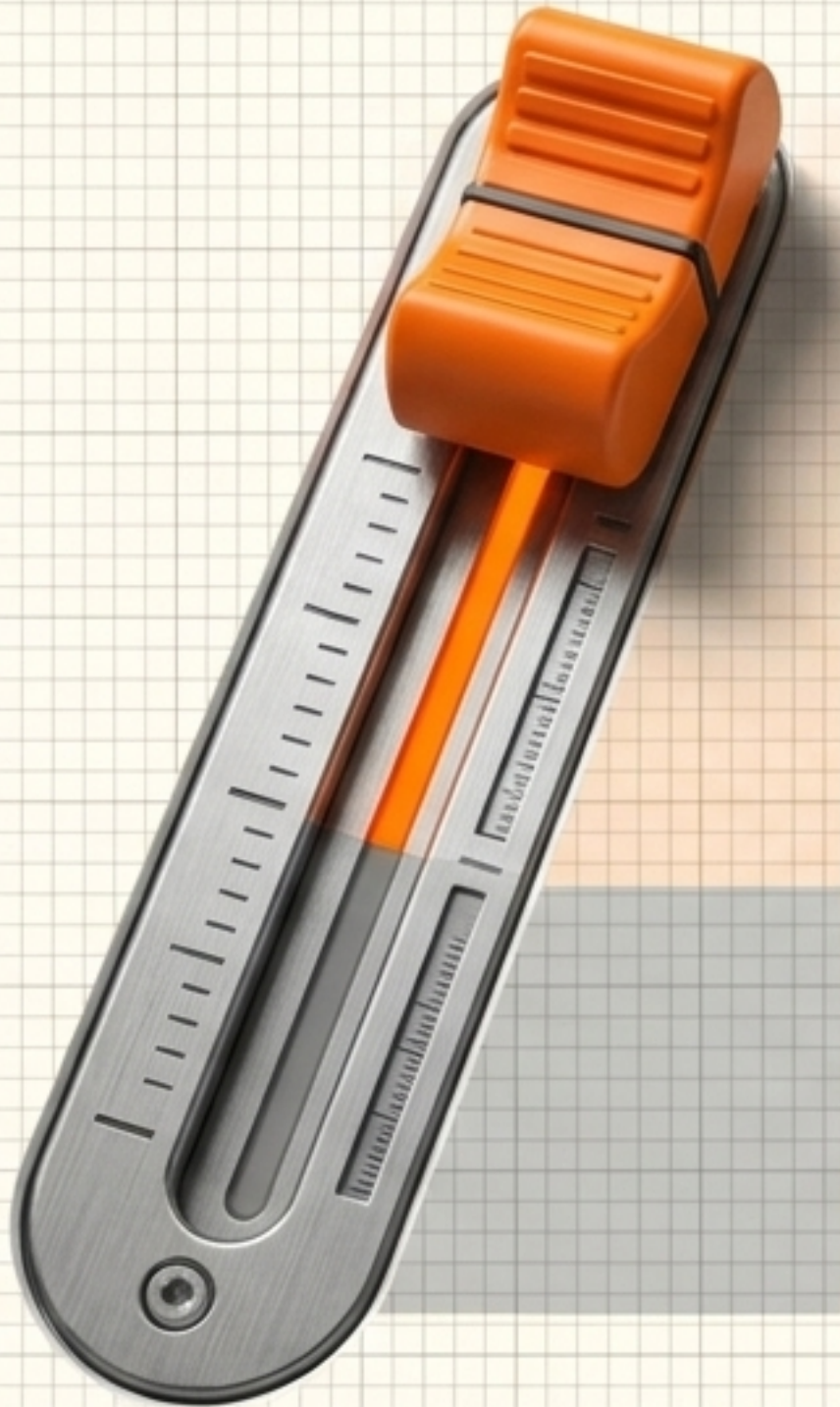
Because AI makes code generation functionally free, our planning medium must shift. Producing throwaway code to answer a question is now faster and more accurate than writing a detailed document describing that same question.



The Planning Medium Matrix

Dimension	Old Way	New Way
Planning Artifact	Text Document	Live Code
Fidelity Level	Low (Concepts & Words)	High (Interactivity & State)
Primary Cost	Human Time	LLM Tokens
Risk of AI Hallucination	High (Ambiguous translation)	Low (Concrete reference)

The Fidelity Spectrum



High Fidelity (**Live Code**)

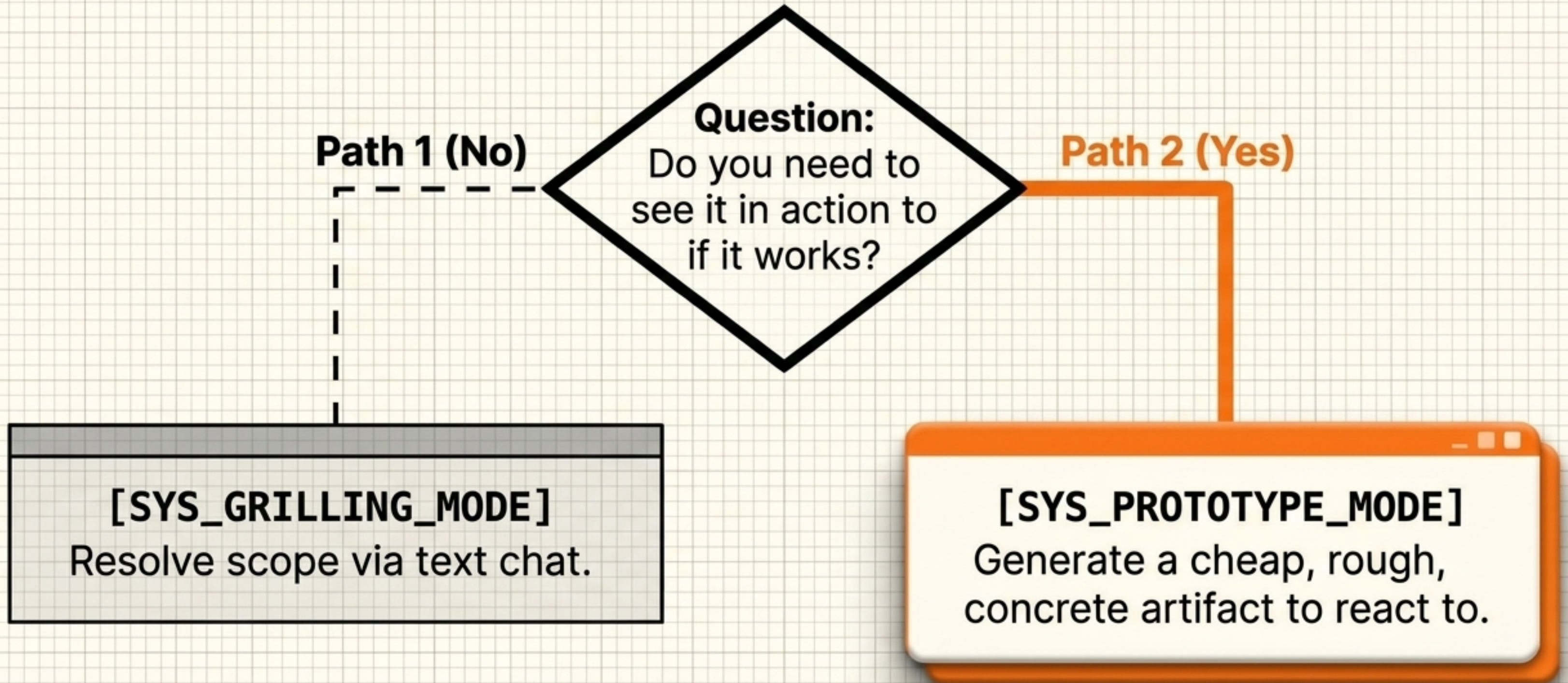
Use for: Complex behavior and feel.

Low Fidelity (**Chat & Grilling**)

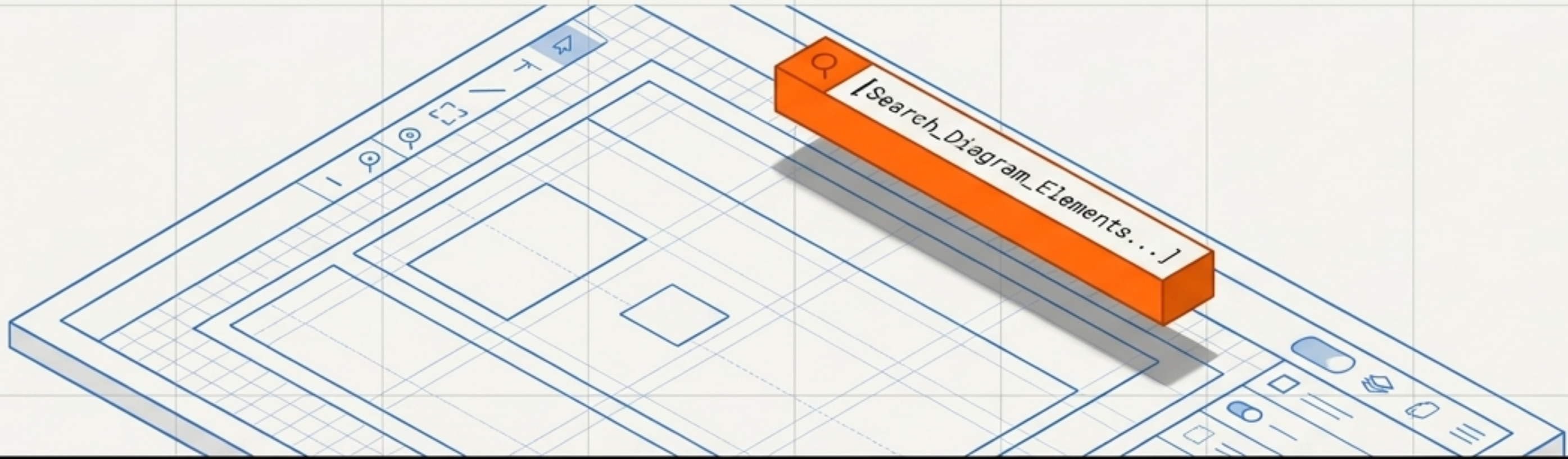
Use for: Basic framing. (e.g., "The modal needs a cancel and confirm button.")

The Trigger Question: Escalate to High Fidelity the moment you ask, "How should this look or behave under specific circumstances?"

The Ticket Typology



The Scenario: Complex State Navigation



Goal: Build a **search bar** for a diagramming app.

The Problem: The data model is **complex** (diagrams + **historical snapshots** over time). Writing a spec for how the search filters should behave is nearly impossible without **feeling it**.

The Solution: Run a **prototyping** session to generate interactive options.

The Exploration Burst

AI generates three distinct interactive variations to react to:

A UI mockup for Option A. It features a search bar at the top with the placeholder text "(Search_Diagrams...)". Below the search bar, there are three sections, each labeled "Diagram Name A", "Diagram Name B", and "Diagram Name C". Each section contains four small, empty rectangular boxes, suggesting a list of items or results grouped by diagram name.

(Good search box placement, poor grouping).

Option A: Reflow in place, rows grouped by diagram name.

A UI mockup for Option B. It features a "FILTERS" sidebar on the left with checkboxes for "Date Range", "Author", "Status", and "Sorting". The "Date Range" section includes a "Date Range" input and a "Between Range" button. The "Author" section includes an "Author" dropdown. The "Status" section includes a "Status" dropdown. The "Sorting" section includes a "Sorting" dropdown. To the right of the filters is a search bar with the placeholder text "Search...". Below the search bar, there are four sections, each labeled "Diagram Name A", "Diagram Name B", "Diagram Name C", and "Diagram Name D". Each section contains a description and a "Current" button.

(Filter resets unexpectedly).

Option B: Left-side grouping, dynamic filtering.

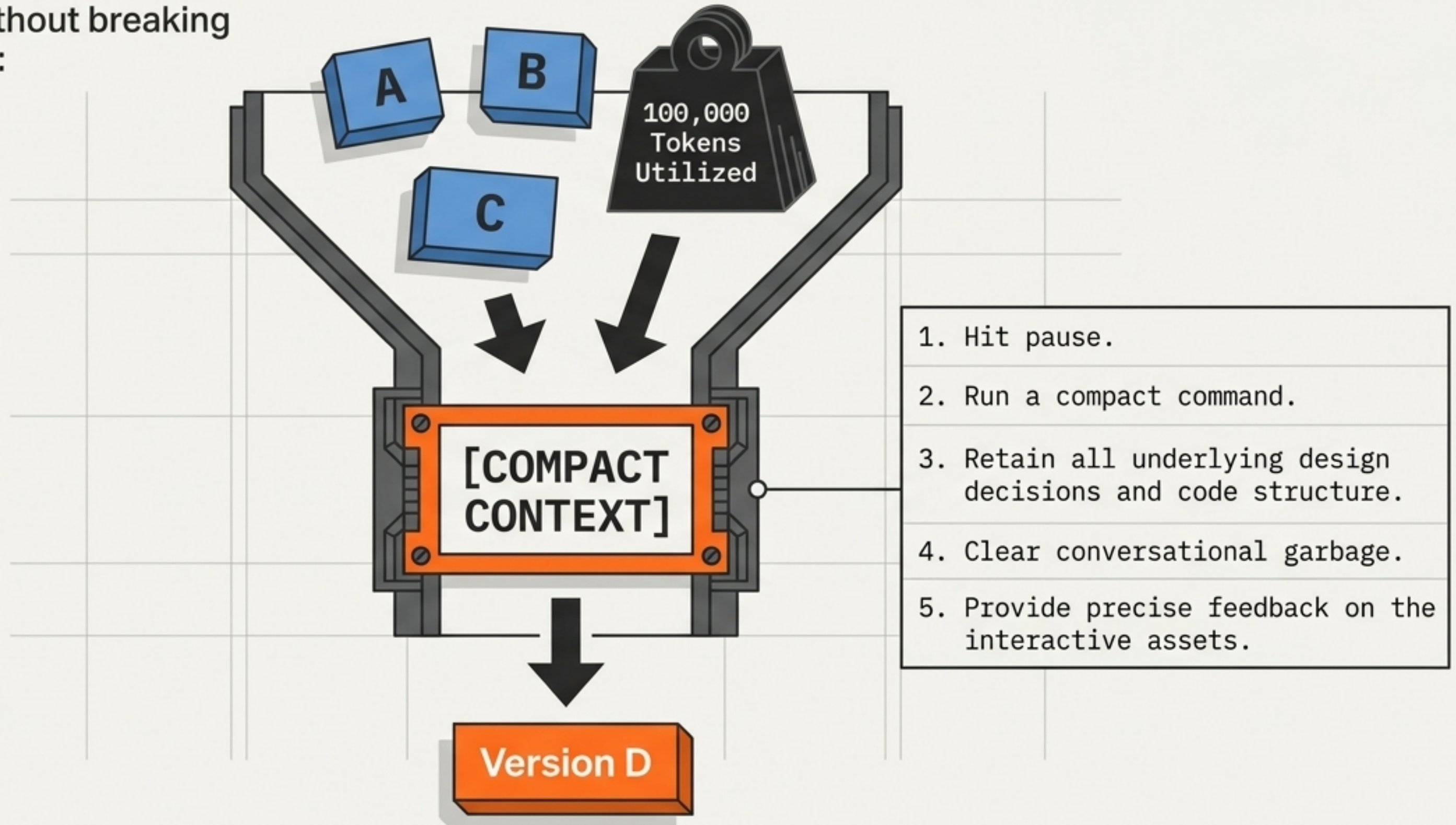
A UI mockup for Option C. It features a search bar at the top with the placeholder text "Search...". Below the search bar, there are four sections, each labeled "Diagram Name A", "Diagram Name B", "Diagram Name C", and "Diagram Name D". Each section contains a description and a "Current" button.

(Clean layout, but shows confusing 'current' snapshot tags).

Option C: Inline results, no filters.

The Compact & Iterate Loop

Prototyping burns tokens fast.
To move forward without breaking
the context window:



Feedback as Code

Human Input: "I really like the search box of A, and the layout of C. Remove the current tags."

Version A [Search Box]

Version C [Layout]

Version D

Version Box

Search...

Search

Layout 1

Layout 2

Layout 3

How today

Cancel

Interactive

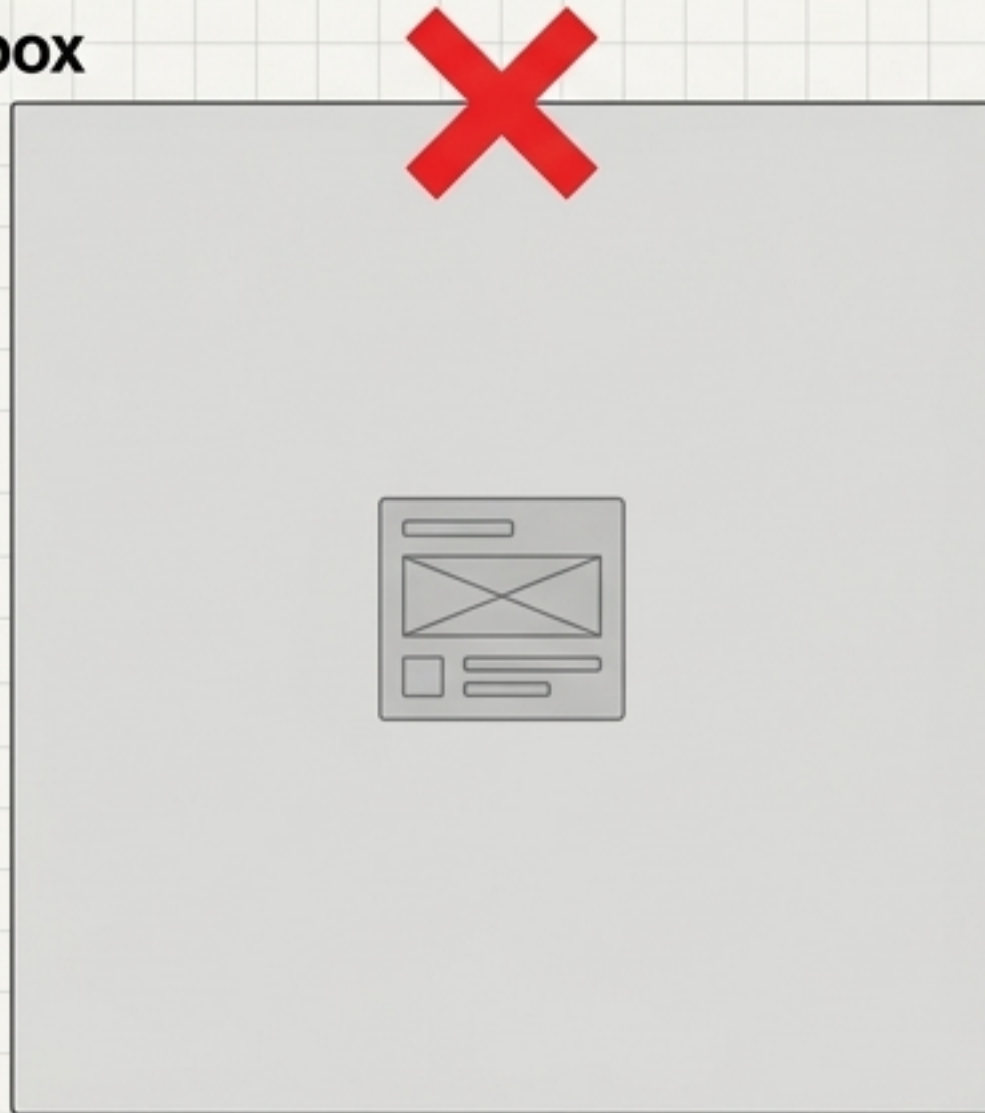
The Output: Version D.

A super-rich asset encoding all refined design decisions into live, front-end code that perfectly matches the user's intent.

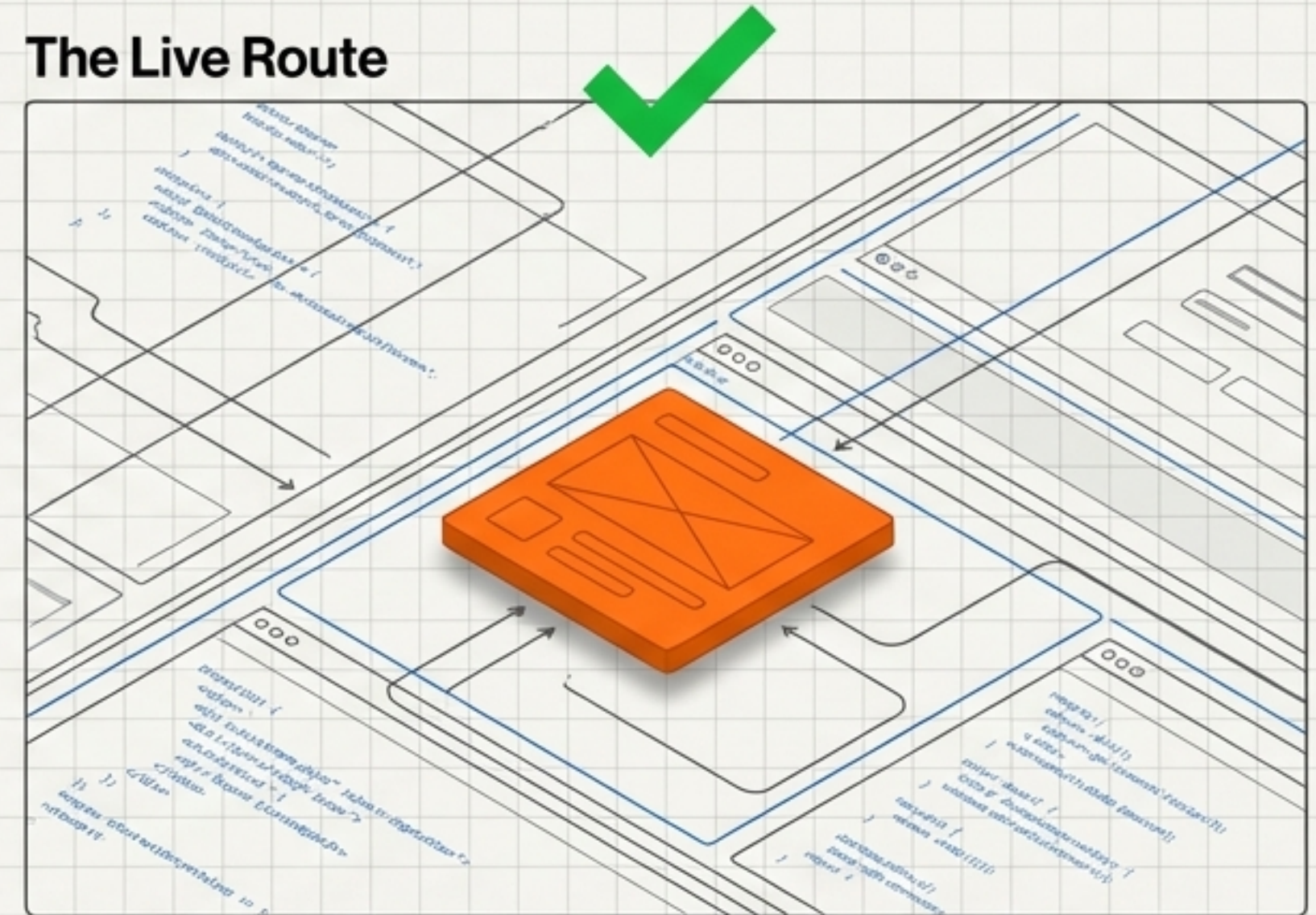
The Live Route Advantage

Do not isolate your prototypes in sandboxes.

The Sandbox



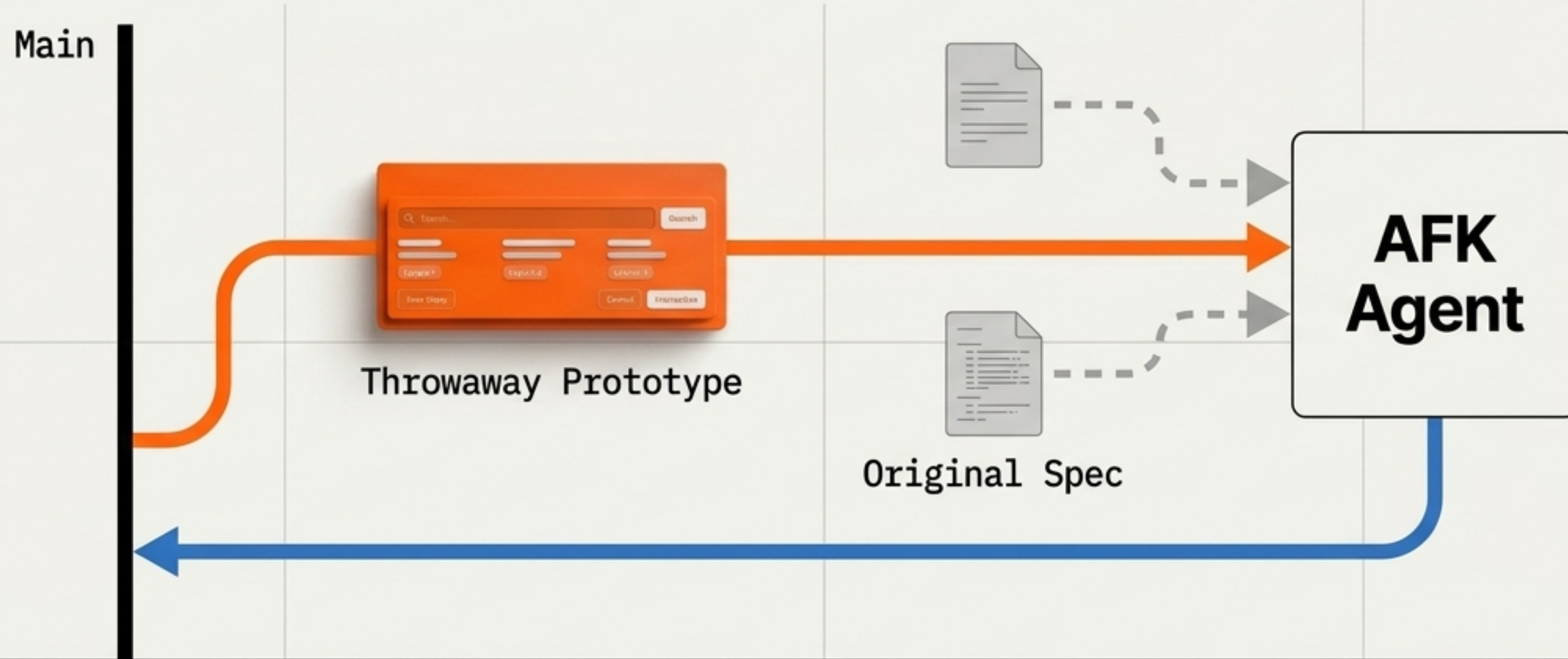
The Live Route



Integrate throwaway code directly into your live application routes. Operating against real data models and live state provides a vastly more honest representation of how the code will actually function in production.

The AFK Agent Handoff

The prototype is a reference artifact, not the final product.



1. Save the prototype on a throwaway branch.
2. Hand off to an AFK (Away From Keyboard) implementation agent.
3. The agent references the live front-end code and the original spec.
4. The agent writes clean, compliant production code and deletes the throwaway logic.

The Prototype Typology Matrix

Visual UI Prototypes

Key Question: "How should it look and feel?"

Asset Generated: Interactive front-end stub.

Benefit: Clarifies user experience and layout logic.



Pure Logic Prototypes >_

Key Question: "Does this state model behave correctly?"

Asset Generated: Tiny interactive terminal app pushing a state machine through edge cases.

Benefit: Validates complex backend logic that is impossible to reason about on paper.

```
[ new .state  
  state => 'pat!  
  raper-n1 => astoHerile  
]  
  
[ enter[...] ]  
> []
```

Testing the Invisible

When building complex backend state models, text specs fail to reveal edge cases.

> RUNNING STATE MACHINE PROTOTYPE

[STATE: IDLE] -> EVENT: FETCH_INIT -> [STATE: LOADING]

[STATE: LOADING] -> EVENT: TIMEOUT -> [STATE: ERROR]

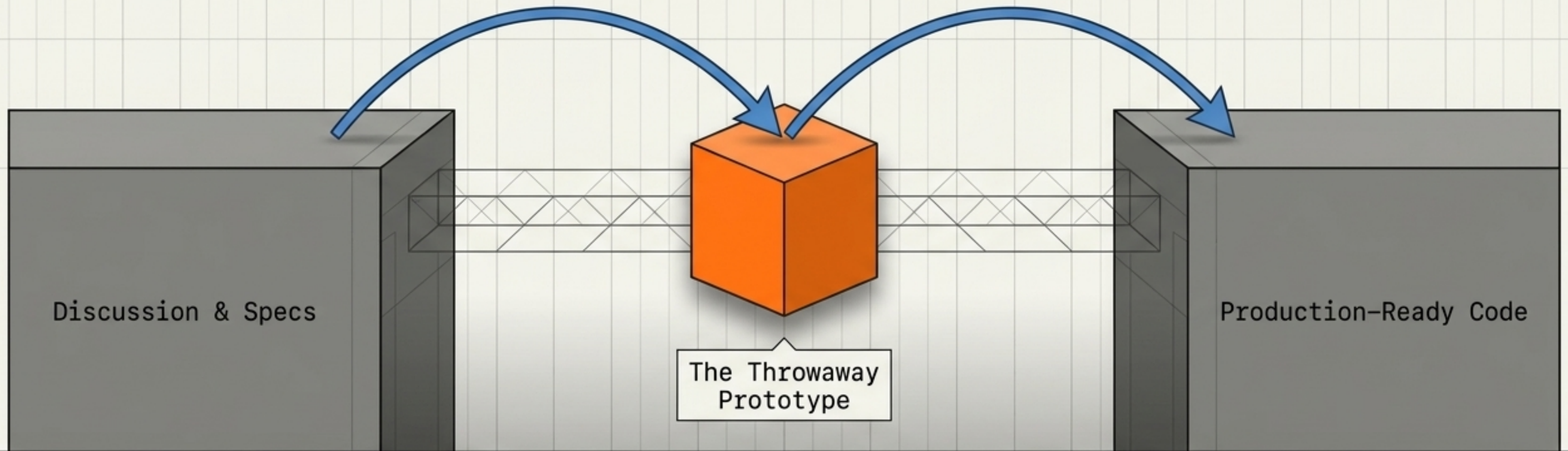
[STATE: ERROR] -> EVENT: RETRY (edge_case=true)

! VALIDATION COMPLETE

Generate a tiny terminal application. Use it to aggressively push your state machine through scenarios that are difficult to visualize on paper.

Answer the critical question: 'Does this logic actually hold up in the real world?'

Bridging the Chasm



The leap from a text document to production-ready AI code is a massive, highly risky gap. AI will often fall short.

Investing tokens into a high-fidelity throwaway prototype places a concrete stepping stone perfectly in the middle. It radically reduces the distance your implementation agent has to jump.



Shape the Work, Write the Code.

Inspired by Ryan Singer's Shape Up, the rule for the AI era is clear:
Force your discussions to a higher fidelity.

Stop expecting AI to perfectly interpret a massive spec. Let
the AI write cheap, disposable code to answer your hardest
questions, and let the prototype become the blueprint.